Aravindh Varadharaj

Full-Stack Engineer · Go · React · TypeScript · Distributed Systems · Data Pipelines

Tiruppur, Tamil Nadu, India · open to relocation, on-site or remote

aravindhvaradha@gmail.com | github.com/Aravindh97 | LinkedIn | aravindh97.github.io/resume

5 full-stack apps
Go on **PostgreSQL** with **React + TypeScript**, each tested in real browsers and load-tested, plus 9 Go systems projects.
Projects and demos at aravindh97.github.io/resume

SUMMARY

Builder at Amazon since 2021 who moved from digital marketing into engineering by automating my team's workflows. I build full-stack products with Go services on PostgreSQL and React + TypeScript frontends: a hospital platform, a personal finance app, a factory order tracker, offline-first notes and a multiplayer game. Each is tested against a real database and in real browsers, load-tested, and run in CI, and I write up the bugs the tests caught.

SKILLS

Backend	Go (goroutines, context, net/http, generics, race detector, benchmarks and profiling), REST APIs, WebSockets, Server-Sent Events, sessions, CSRF protection, bcrypt
Frontend	React, TypeScript, Vite, React Router, TanStack Query, react-hook-form, zod, accessible responsive UI, service workers, IndexedDB, Go compiled to WebAssembly, types generated from Go structs
Data	PostgreSQL (schema design, transactions, row locks, SKIP LOCKED, advisory locks, partial unique indexes, triggers, LISTEN/NOTIFY, logical replication, pgvector), OpenSearch
Reliability	idempotency keys, double-entry ledgers, optimistic concurrency, retries with backoff, CRDTs, Temporal workflows, kill -9 and outage tests, load testing
Testing & delivery	Go integration tests on real Postgres, Vitest, Playwright with axe accessibility checks, GitHub Actions, Docker, Render
AI coding tools	Kiro, Claude Code and Copilot daily; concurrency, SQL and money logic reviewed by hand

PERSONAL PROJECTS

Built in my own time to learn; not affiliated with my employer. Live demos and source at github.com/Aravindh97.

Kaasu · Personal Finance App (Wallet, Statement Analyser, Budgets)

Live demo · GitHub

Go · PostgreSQL · WebAssembly · React · TypeScript · TanStack Query · Playwright · React + TypeScript frontend

- Double-entry ledger in paise with idempotency keys written in the payment's own transaction: under load, 431 payments retried with the same key were never paid twice, and the ledger balanced after 10,306 requests.
- Payment PIN that locks after 3 tries, daily limits, and a fraud rule that pauses an account after 5 payments to new payees in 10 minutes, with an ops review queue and an append-only audit log.
- One Go statement analyser runs on the server or as WebAssembly in the browser (privacy mode saves only the summary); it finds subscriptions, price rises, bank charges and double payments, and learns from corrections.

Arogya · Hospital Platform (OPD Tokens, Records, Beds, Blood Bank)

Live demo · GitHub

Go · PostgreSQL · React · TypeScript · Server-Sent Events · Playwright · React + TypeScript frontend

- Contested slots, beds and blood units settled in the database (unique indexes, SKIP LOCKED in expiry order); tests race 20 receptionists for one slot and 10 emergency requests for scarce O- units: never double-allocated.
- Records open only through a care relationship or a logged break-glass reason; every access goes to an append-only log the patient can read. The public token board shows numbers only.
- 41,874 requests in 20 s (about 2,100/s), walk-in token p99 61 ms. The load test caught duplicate record numbers under concurrent registration (half failed); fixed with a Postgres sequence.

Thari · Garment Order Tracker for Factories and Buyers

Live demo · GitHub

Go · PostgreSQL · React · TypeScript · TanStack Query · react-hook-form · zod · React + TypeScript frontend

- Permissions applied inside every query: buyers see only their own orders, never prices or internal notes; live updates through Postgres LISTEN/NOTIFY and Server-Sent Events.
- A concurrency test caught a lost-update bug (40 updates accepted, only 8 saved); fixed by locking before reading. Edits use optimistic versions; the audit log is append-only.
- 50 users: about 3,700 requests/s with p99 under 26 ms; 17 Go integration, 7 Vitest and 8 Playwright tests with axe.

Kurippu · Offline-First Notes with a Go CRDT

Live demo · GitHub

Go · WebAssembly · PostgreSQL · IndexedDB · Service Worker · React · TypeScript · React + TypeScript frontend

- One Go text CRDT (RGA) runs on the server and in the browser as WebAssembly; 1,500 random editing sessions converge, and in Playwright two offline browsers' edits to the same note both survive.
- Ordered per-notebook op log in Postgres with idempotent uploads; works with no network through a service worker and IndexedDB. A keystroke takes 0.12 ms at 1,000 characters, 1.2 ms at 10,000.

Pallanguzhi · Online Multiplayer Board Game — Live demo · GitHub

Go · WebSockets · WebAssembly · React · TypeScript · React + TypeScript frontend

- Server-authoritative board with versioned moves; seats survive reloads and dropped networks; spectators and rematch. 1,000 games over 4,000 WebSocket connections with 0 failures; the load test caught slow clients being dropped.
- Go alpha-beta AI that explains its moves; a reproducible 2,800-game benchmark showed the second player has a large advantage, so results are reported by seat.

GO SYSTEMS PROJECTS

Each with tests, CI and a live demo of its Go code compiled to WebAssembly.

- **granth**: Document ingestion and hybrid retrieval on Temporal and OpenSearch; survived an OpenSearch outage mid-ingest with no duplicates; hit@5 0.977 vs 0.955 for BM25.
- **smriti**: Long-term memory for LLM agents on pgvector; 1.000 correct and 0 stale answers vs 0.676 and 0.176 for raw-history retrieval.
- **sutra**: Postgres CDC connectors and entity resolution; source and warehouse matched exactly after kill -9; F1 up to 0.996 vs 0.77.
- **fairdrop**: Ticketing for high-demand releases; 50,000 simulated buyers, 0 seats sold twice vs 156,874 extra bookings naively.
- **paisa**: Exactly-once ledger; books balanced through 20 kill -9s in 50,000 payments; sharding the hot account raised throughput 6.8x.
- **orbit**: Satellite conjunction screener; 16,763 objects over 6 days in 3 min 49 s, 190 of 190 brute-force encounters found.
- **relief**: Offline sync for relief teams with CRDTs and hybrid logical clocks; 10,000 randomised runs, all converged.
- **aqcal**: Air-sensor calibration and fault detection; 6.9 M readings/s, PM2.5 error cut 52% on held-out days.
- **foodwatch**: Food-stress early warning for Somalia; flagged 20 of 26 crisis onsets a median 21.6 weeks early.

EXPERIENCE

Sr. Digital Marketing Associate, Amazon — 2021 to Present

- Built and automated internal tools and workflows for my team, replacing repetitive manual tasks with reliable, reusable solutions.
- Worked with stakeholders to find process gaps, then designed, built and supported automation that the team adopted in daily work.
- Grew from a marketing role into hands-on development, and built full-stack Go and React + TypeScript applications outside my core responsibilities.

EDUCATION

Bachelor of Engineering (B.E.) in Mechanical Engineering